Vision-Based Misalignment Diagnosis of the SOFIA Units: From a First Run to a Larger Dataset

Marco Heimdal Castro Magaña, Alejandro Amador Franco, Karen Alejandra Guevara Vasquez, María Alicia Vela Contreras, Juan Pablo Contreras Quiñones, Jorge A. Lizarraga, Luis E. González-Jiménez, Luis F. Luque-Vega

***Abstract*—A SOFIA module has 16 hexagonal plates, and each one can tilt in two directions [1]. This work estimates the two angles of every plate from one overhead photograph and labels each plate as OK ($< 2^\circ$), REVIEW (2–5°), or MISALIGNED ($> 5^\circ$). Three methods were compared on images generated in Webots: a geometric method without training, a small convolutional neural network (CNN), and a hybrid rule that combines them. A first run used simplified plates and 2000 images; a second run used the simulation model of the real unit and 10 000 images. In the second run, the CNN reached a mean error of 0.20° (before, 0.25°) and became the best method in every range, while the geometric method and, with it, the hybrid rule lost accuracy because the plate of the real unit moves less per degree. The algorithms are explained as state machines.**

***Index Terms*—Misalignment diagnosis, ArUco markers, convolutional neural networks, Webots, synthetic data, state machines.**

I. How the Samples Were Obtained

A. The Photograph

Four ArUco markers [2] are placed around the module, at the height of the plates. From their centers, the photograph is *rectified*: it is converted into a top view where 1 px equals 1 mm, no matter where the camera is. In that view, every plate at zero always falls on the same pixel, so a square crop of 128×128 px is cut around each plate. The 16 crops are what the methods receive.

B. The Generator in Webots

Taking thousands of real photographs with known angles is not practical, so the photographs were generated in Webots [3], where the angles are known exactly. A supervisor controller and the 16 plate controllers follow the state machine of Fig. 1. For each sample, the supervisor chooses random angles and a random scene (camera height of 1.30–1.40 m, tilt of up to $\pm 4^\circ$, yaw of up to $\pm 8^\circ$, lights, and floor), sends the angles, and waits until all 16 plates report that they are still. Only then it takes the photograph and saves, as the label, the angles *measured* by the joint sensors, not the commanded ones. If a plate does not arrive within 3 s, the sample is discarded. Sample 0, with every plate at zero, is the reference.

C. First Run and Second Run

Table I compares the two runs. The second run replaced the simplified plates with the PROTO of the real unit (generated from its CAD model with `urdf2webots` [4]), placed the units as close as possible without collisions, moved the markers closer, used five times more images, and changed how the angles are chosen: in the first run they were uniform, while in the second they follow what is expected in the robot, mostly small misalignments and sometimes large ones, as if a plate were moved by hand or jammed. In both runs the images were split *by photograph* into training, validation, and test sets, so the 16 plates of one photograph never fall in different sets.

The authors are affiliated with the Instituto Tecnológico de Estudios Superiores de Occidente (ITESO), Tlaquepaque, Jalisco, Mexico.

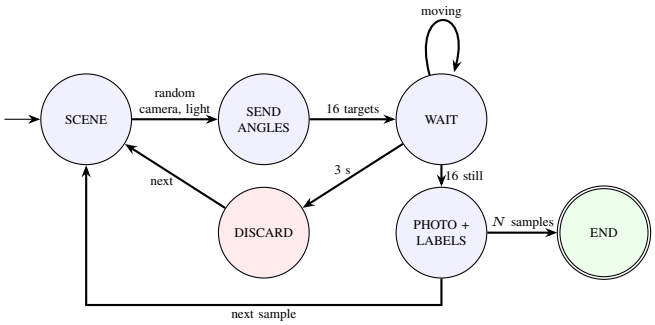


Fig. 1. State machine of the dataset generator. A photograph is taken only when the 16 plates report that they reached their target.

TABLE I
The Two Runs

	First run	Second run
Unit	Simplified plate, 69 mm, arm $L = 58$ mm	Real unit (PROTO), 60 mm, arm $L = 37$ mm
Markers	0.76 m square	0.60 m square
Images (train / val / test)	2000 (1399 / 300 / 300)	10 000 (6999 / 1500 / 1500)
Angles per plate	5 % zero, 50 % within $\pm 3^\circ$, 45 % within $\pm 27^\circ$	10 % zero, 60 % up to 5°, 20 % 5–15°, 10 % 15°–limit
CNN training	40 epochs, 84 min	25 epochs, ≈4 h, in parts

II. Methods and Training

A. Geometric Method

It finds each plate in the top view and measures how far its center moved from the reference. With the plate at a distance L from the joint axes, the shift is $\Delta v = L \sin\theta_2$ and $\Delta u = L \cos\theta_2 \sin\theta_1$, which gives the two angles directly. It needs no training, but an error of 1 mm in the position becomes an error of $1/L$ radians: 1.0° with $L = 58$ mm and 1.5° with $L = 37$ mm.

B. CNN

A small CNN (five blocks of convolutions, 1.28 M parameters) receives one crop and returns the two angles. It

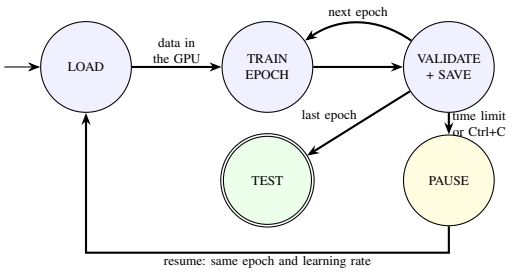


Fig. 2. State machine of the training. The state is saved after every epoch, so training can be split into several sessions.

was trained in PyTorch [5] with the training images, mirrored and with random changes of light and noise so that it does not depend on a single scene. After every epoch, its error is measured on the validation images, and only the best epoch is kept. The test images are used once, at the end, to report the results.

Since the second run takes about 4 hours, training was split into sessions with the state machine of Fig. 2. The complete state (weights, optimizer, and learning rate) is saved after every epoch, so a session can stop after a time limit or with Ctrl+C and the next one resumes at the same epoch, as if it had not stopped.

C. Hybrid Rule

Both methods are run, and for each plate: if they differ by more than 2°, the CNN is used and the plate is marked for inspection; if they agree and the plate is near zero ($\leq 6°$), the geometric value is used; otherwise, the CNN. It was designed in the first run, where the geometric method was the most accurate near zero.

D. Why ResNet18 Was Dropped

In the first run, a ResNet18 [6] pretrained on ImageNet was also trained. Although it is ten times larger (13.3 M parameters), it was less accurate than the small CNN (Table II): what it learned from everyday photographs does not help to measure a shift of a few pixels. For this reason, and to save training time, it was not used in the second run.

III. Results

Table II shows the error on the test images of each run (300 and 1500 photographs). The two test sets are not the same, since the unit and the distribution of angles changed, so the comparison is between the two complete setups.

First run. Each method was good at something different. The geometric method was the most precise near zero ($0.13°$), but sometimes failed badly (up to $74°$) when a very tilted plate was confused with its neighbor. The CNN never failed badly (at most $4.4°$). The hybrid rule took the best of both and was the best method ($0.223°$).

Second run. The CNN improved to $0.204°$ with an error of at most $3.3°$ in 24 000 plates, and became the best method in every range, even near zero. The geometric method lost precision near zero ($0.13° \rightarrow 0.33°$) because the arm of the real unit is shorter, so the plate moves less per degree and each pixel of error weighs more; on the other hand, it no longer fails badly (at most $5.9°$). As a consequence, the hybrid rule *fell* from the best to the middle ($0.329°$): it uses the geometric value precisely near zero, where it is now the least precise. Its inspection marks are not useful either: 873 of its 901 marks were errors of the geometric method, not of the CNN. The CNN is therefore the default method; the geometric method remains as a fallback without a GPU.

Two lessons follow. First, more and more realistic images made the CNN better, even with a harder unit. Second, a CNN must be retrained when the unit changes: the CNN of the first run, tested on the real unit, had an error of $1.6°$.

TABLE II
Test Error (°) and State Accuracy

Method	Mean	Max	0–5°	> 15°	States
First run (simplified plates, 2000 images)					
Geometric	0.539	74.0	**0.13**	1.30	97.2 %
CNN	0.248	4.4	0.18	**0.36**	96.3 %
ResNet18	0.293	4.1	0.21	0.43	95.9 %
Hybrid	**0.223**	4.4	**0.13**	**0.36**	**97.2 %**
Second run (real unit, 10 000 images)					
Geometric	0.521	5.9	0.33	1.60	94.1 %
CNN	**0.204**	**3.3**	**0.17**	**0.41**	**96.8 %**
Hybrid	0.329	**3.3**	0.33	**0.41**	94.1 %

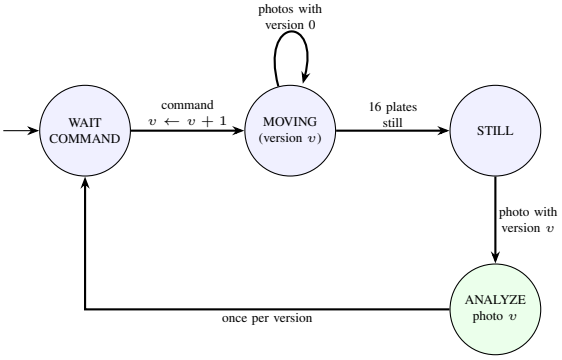


Fig. 3. State machine of the live mode. Each command produces exactly one diagnosis, with a photograph taken after the plates stopped.

IV. Diagnosis Application

The methods are used in an application with its own processing thread, which can read a camera, saved photographs, or a live Webots simulation. In the live mode, the application moves the simulated plates and checks each diagnosis against the measured angles. The difficulty is to analyze the right photograph: one taken after the plates stopped, and only once per movement. This is solved with the state machine of Fig. 3: every command increases a version number, and a photograph carries that version only when the 16 plates are still.

V. Conclusions

One photograph with four markers is enough to estimate the angles of the 16 plates. In a first run, a hybrid rule was the best method because the geometric method and the CNN failed in different places. With the real unit and five times more realistic

images, the CNN became the best method everywhere (0.20° of mean error, 3.3° at most, and $96.8\,\%$ of correct states), and the hybrid rule stopped being useful, since the shorter arm of the real unit penalizes the geometric method. ResNet18 was dropped because a larger network did not help. The next step is to test the CNN with photographs of the physical module and fine-tune it with a few real images.

REFERENCES

[1] J. A. Lizárraga, "Presentación del labdst: Sesión 1 - contexto, metodología d-lab y sofia," 2026, iTESO, 17 de agosto de 2026.

[2] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.

[3] O. Michel, "Cyberbotics ltd. Webots: Professional mobile robot simulation," *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 39–42, 2004.

[4] Cyberbotics Ltd., "urdf2webots: Convert URDF robot descriptions to Webots PROTO nodes," 2025. [Online]. Available: https://github.com/cyberbotics/urdf2webots

[5] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.